

DUOC UC

Escuela de Informática y Telecomunicaciones

Estado de Avance N°1

Evaluación Parcial N° 1

Plataforma Cloud de Gestión de Mantenimiento para Taller Minero

Asignatura: TPY1101 — Taller Aplicado de Programación

Sección: 002v

Grupo: 10

Integrantes:

**ROSIO CATALINA AMETLLER VARGAS
RAMON ANDRES HERNANDEZ CORRALES
JAIME GUILLERMO OSORIO URBINA**

2026

Índice

Índice	2
Introducción.....	4
1. Contexto del proyecto y modelo de solución	5
1.1 Contexto operacional del proyecto	5
1.2 Necesidad de una solución informática cloud	6
1.3 Modelo preliminar de solución	6
1.4 Procesos de negocio afectados por la solución	7
2. Problema u oportunidad del entorno	8
3. Objetivo general y objetivos específicos	10
3.1 Objetivo general	10
3.2 Objetivos específicos	10
4. Alcance del proyecto	11
4.1 Alcance general	11
4.2 Elementos incluidos dentro del proyecto.....	11
4.3 Componentes considerados pero no implementados completamente en esta etapa .	11
4.4 Exclusiones reales del proyecto	11
4.5 Entregables del proyecto	11
4.6 Supuestos	12
4.7 Restricciones.....	12
4.8 Relación del alcance con los objetivos.....	12
5. Planificación de actividades	13
5.1 EDT / WBS del proyecto	13
5.2 Asignación de responsables por rol	13
5.3 Carta Gantt simplificada por semanas	13
5.4 Secuencia lógica de trabajo.....	13
6. Determinación de tecnologías y servicios cloud	15
6.1 Stack tecnológico seleccionado.....	15
Frontend — Next.js sobre Vercel (PaaS).....	15
Backend — NestJS sobre Railway (PaaS).....	15
Base de datos y servicios complementarios — Supabase (BaaS/PaaS)	15
6.2 Clasificación de servicios cloud utilizados.....	16
7. Justificación de la elección tecnológica	17
7.1 Por qué cloud y no on-premise.....	17
7.2 Justificación de cada componente del stack	17
Next.js + Vercel — Justificación del frontend	17
NestJS + Railway — Justificación del backend	17
Supabase — Justificación de la base de datos	17
Linear — Justificación de la gestión de proyecto	18

7.3 Factibilidad de cumplimiento	18
8. Conceptualización de la solución	19
8.1 Propósito de la solución	19
8.2 Valor que agrega al cliente	19
8.3 Atributos de calidad de la solución	19
Integridad	19
Confiabilidad	20
Precisión	20
Oportunidad	20
Seguridad	20
8.4 Viabilidad de la solución	21
9. Metodología, ciclo de vida, patrón de diseño y mecanismos de control	22
9.1 Metodología seleccionada: Scrum adaptado al contexto académico	22
Adaptaciones al contexto académico	22
9.2 Ciclo de vida del proyecto: Iterativo-incremental	22
9.3 Patrón de diseño adoptado: Arquitectura modular por capas	23
9.4 Mecanismos de control y gestión del proyecto	23
10. Documentación, revisiones y estándares de calidad	25
10.1 Enfoque de documentación del proyecto	25
10.2 Estructura documental por entrega parcial	25
10.3 Registro de revisiones y certificación de avance	25
10.4 Estándares de calidad aplicados	26
10.5 Trazabilidad entre documentación y entregables	26
10.6 Gestión de la documentación	27

Introducción

El presente documento constituye el Estado de Avance N°1 del proyecto de diseño de una solución informática cloud orientada a la gestión integral del proceso de mantenimiento de equipos en un taller minero. La propuesta nace de una problemática concreta: la fragmentación operacional y la falta de trazabilidad en tiempo real que afectan la coordinación entre las áreas de mantenimiento, bodega, abastecimiento y administración de sistemas.

El informe aborda diez dimensiones fundamentales del proyecto. En primer lugar, se presenta el contexto operacional y el modelo preliminar de solución, identificando los procesos de negocio afectados. Luego se analiza el problema u oportunidad del entorno, se formulan los objetivos general y específicos, y se define el alcance del proyecto con sus entregables, supuestos y restricciones. A continuación, se presenta la planificación de actividades mediante EDT y carta Gantt. Finalmente, se desarrollan las secciones de determinación tecnológica, justificación de la elección cloud, conceptualización de la solución con atributos de calidad, definición de metodología y ciclo de vida, y documentación asociada a revisiones y estándares de calidad.

El objetivo de este entregable es establecer la línea base del proyecto: definir qué se va a hacer, por qué, con qué tecnología y bajo qué condiciones, demostrando la viabilidad técnica, operativa y económica de la solución propuesta.

1. Contexto del proyecto y modelo de solución

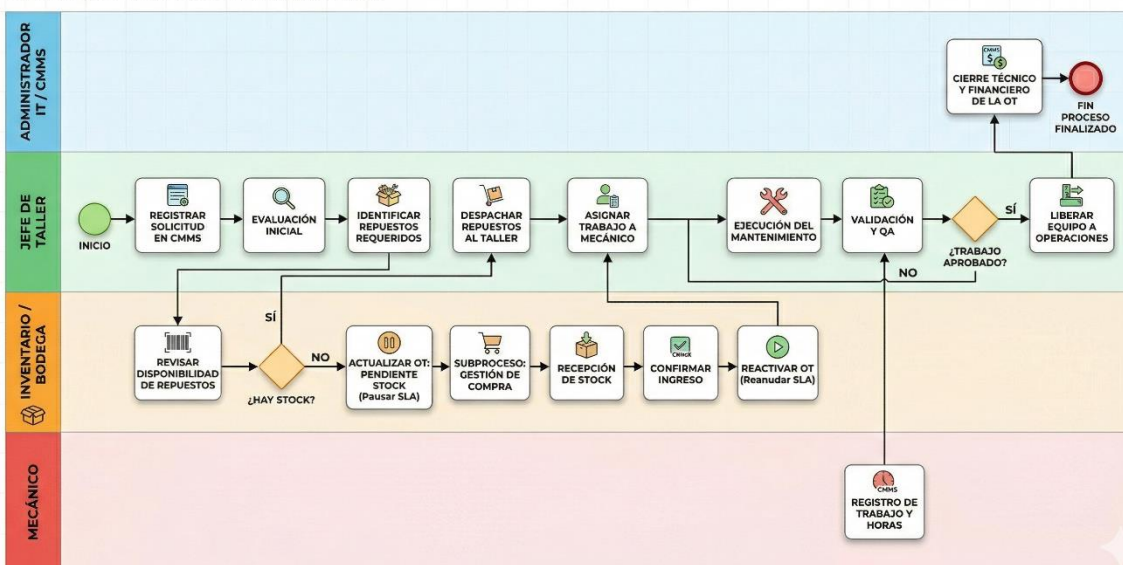
1.1 Contexto operacional del proyecto

Este proyecto se desarrolla en el contexto de la mantención de equipos dentro de un taller minero, un entorno donde la continuidad operacional depende de responder con rapidez y orden frente a fallas, mantenimientos correctivos y actividades programadas, sin comprometer la disponibilidad de activos críticos. El cliente real puede ser una operación minera, una gerencia de mantenimiento o taller de una compañía del rubro, o incluso un contratista especializado que presta servicios sobre flota y equipos de apoyo. En ese marco, la solución apunta a operaciones con alto volumen de órdenes de trabajo, fuerte dependencia de repuestos, exigencias de trazabilidad y necesidad de coordinación entre mantenimiento, bodega, abastecimiento y administración de sistemas.

El flujo actual ya tiene una lógica definida. Todo parte con la generación de una Orden de Trabajo en el CMMS. Luego viene la evaluación inicial del equipo, la identificación de repuestos necesarios y la revisión de stock por parte de bodega. Cuando no hay disponibilidad, la OT cambia a estado pendiente, se pausa el SLA y se activa el subproceso de compra. Una vez que llegan los materiales, el trabajo se asigna al mecánico, quien ejecuta la intervención bajo protocolo LOTO, registra horas y actividades, y finalmente deja el trabajo listo para revisión del Jefe de Taller, quien valida la calidad de la intervención y autoriza el cierre técnico y financiero. A simple vista parece un flujo operativo. En realidad, es un proceso transversal, lleno de trasposos entre áreas y decisiones que afectan tiempos, costos y riesgo.

Cuando una OT se extiende más de la cuenta, el problema no queda encerrado en mantenimiento. Golpea la programación, afecta la utilización de personal, tensiona la disponibilidad de repuestos y termina impactando la productividad del negocio. Por eso, el proceso no necesita solo un sistema que registre eventos. Necesita una solución capaz de ordenar el flujo, dejar trazabilidad completa, sincronizar información entre áreas y entregar visibilidad real para decidir a tiempo.

DIAGRAMA DE FLUJO: PROCESO DE MANTENIMIENTO INDUSTRIAL (CMMS)
GESTIÓN DE ORDEN DE TRABAJO (OT)



1.2 Necesidad de una solución informática cloud

El caso plantea la existencia de un CMMS, pero el problema no parece estar en la falta total de software. El verdadero quiebre está en la fragmentación del proceso. En muchas operaciones, la OT se genera en el sistema, sí, pero el resto del seguimiento sigue resolviéndose con registros dispersos, comunicaciones informales o controles paralelos. Ahí se pierde visibilidad. También se vuelve más difícil medir tiempos reales, seguir estados con consistencia y distinguir cuándo una orden está detenida por una causa justificada, como la falta de repuestos.

En ese escenario, una solución cloud tiene sentido porque permite articular el proceso completo desde una sola plataforma. No solo centraliza datos. También facilita la operación web y móvil, habilita trazabilidad en línea, mejora la gestión de usuarios y roles, permite integraciones vía API y hace posible desplegar la solución por etapas sin obligar, al menos en una primera fase, a reemplazar de golpe los sistemas corporativos existentes. Dicho simple: no se trata de sumar otra herramienta. Se trata de ordenar un proceso que hoy funciona, pero lo hace con demasiada fricción.

1.3 Modelo preliminar de solución

La propuesta considera una plataforma cloud de gestión de mantenimiento operativo integrada al ecosistema CMMS/ERP de la organización. No busca sustituir de inmediato al sistema corporativo actual. Su rol inicial sería más inteligente que eso: actuar como una capa de orquestación, trazabilidad, seguimiento y apoyo a la toma de decisiones para el proceso específico del taller minero.

Su diseño preliminar contempla los siguientes módulos:

Gestión de OT y estados del proceso. Permitirá crear, visualizar, actualizar y seguir órdenes de trabajo mediante estados normalizados como creada, en evaluación, pendiente por stock, en espera de compra, asignada, en ejecución, en validación, en retrabajo, liberada y cerrada. Cada transición deberá dejar registro de responsable, fecha, observaciones y evidencia.

Planificación y evaluación técnica. Este módulo ordenará la evaluación inicial del Jefe de Taller, la identificación de repuestos requeridos, la criticidad de la intervención, la prioridad del trabajo y las condiciones previas necesarias para ejecutar un mantenimiento seguro.

Inventario y disponibilidad de repuestos. Permitirá consultar stock, reservar materiales para una OT, registrar faltantes y vincular el estado del repuesto con el avance real del mantenimiento. Si no existe disponibilidad, el sistema deberá gatillar alertas y actualizar el estado de la orden sin depender de llamadas, mensajes o seguimiento informal.

Abastecimiento e integración con compras. Su foco no será administrar toda la cadena de compras de la empresa, sino dar visibilidad al evento de compra requerido, al estado del requerimiento y a la fecha estimada de reposición, de modo que mantenimiento pueda operar con información útil y no a ciegas.

Control de SLA y tiempos operacionales. Deberá distinguir entre tiempo activo de intervención y tiempo detenido por causas justificadas, como espera por stock. Esa diferencia no es menor. Sin ella, cualquier medición de desempeño termina contaminada.

Asignación y ejecución técnica. Permitirá asignar trabajos según disponibilidad, especialidad o carga de trabajo del mecánico, y registrar ejecución, protocolos de seguridad aplicados, horas hombre, hallazgos y evidencia técnica.

Validación, QA y retrabajo. Aquí el Jefe de Taller podrá validar la calidad de la intervención bajo criterios trazables. Si el trabajo no cumple, el sistema abrirá el circuito de retrabajo sin perder historial ni romper la integridad de la OT.

Cierre técnico-financiero y reportería. Este módulo consolidará materiales, tiempos, responsables, observaciones y estado final, dejando respaldo para el cierre de la orden y para la generación de indicadores de gestión.

1.4 Procesos de negocio afectados por la solución

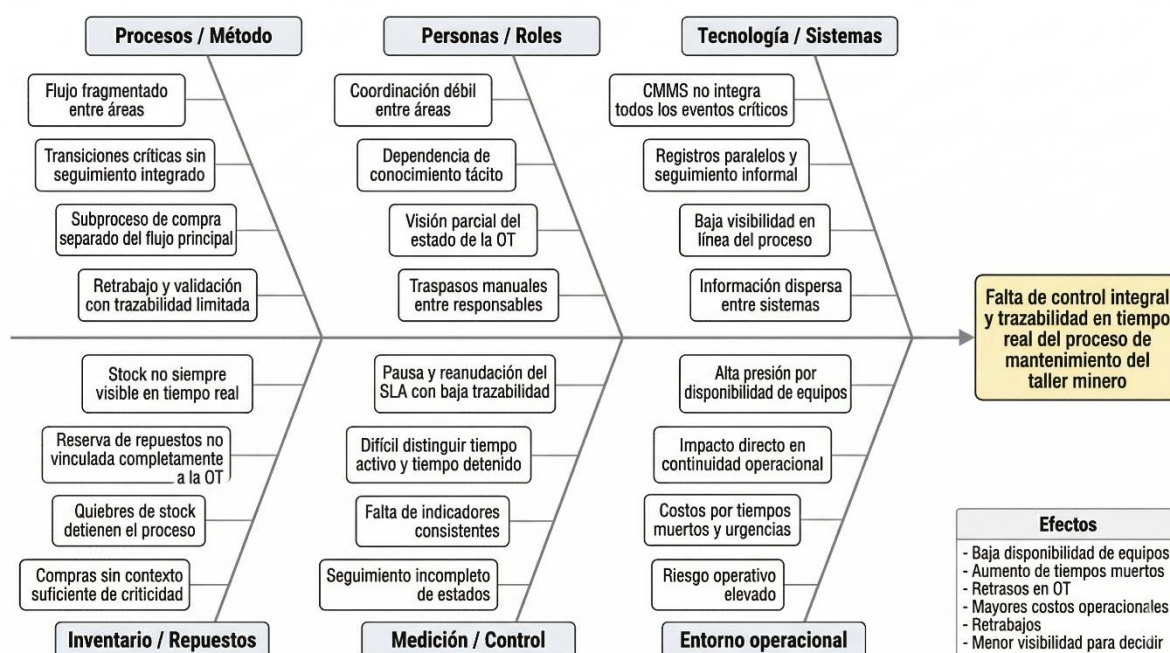
La solución impacta de forma directa la gestión de órdenes de trabajo, porque introduce estados estandarizados, responsables identificables y control temporal desde la apertura hasta el cierre. También mejora la evaluación y planificación del mantenimiento, al formalizar el diagnóstico inicial y las condiciones previas que deben cumplirse antes de intervenir un equipo.

La gestión de repuestos deja de ser un chequeo aislado y pasa a convertirse en una parte crítica del flujo. Lo mismo ocurre con abastecimiento y compras, que dejan de operar como un carril separado y pasan a integrarse como una etapa visible del proceso de mantenimiento. Por su parte, el control de SLA gana profundidad, porque ya no mide solo cumplimiento bruto: permite separar tiempos efectivos de trabajo de tiempos detenidos por causas externas. La ejecución técnica también mejora, ya que se fortalece el registro en terreno, la captura de evidencia y la consistencia del dato generado por el mecánico. Finalmente, el cierre técnico y financiero se vuelve más robusto al consolidar toda la trazabilidad en una sola línea operativa. Ese punto es clave. Sin una visión compartida del proceso, cada área termina trabajando con su propia versión de la realidad.

2. Problema u oportunidad del entorno

La principal problemática del entorno analizado está en la falta de control integral y trazabilidad en tiempo real del proceso de mantenimiento del taller minero, sobre todo en los puntos donde se cruzan la gestión de OT, la disponibilidad de repuestos, el abastecimiento, la ejecución técnica y la validación final. El flujo existe. Los responsables también. Pero eso no basta para decir que el proceso está realmente gobernado. Cuando la información queda repartida entre distintos actores, registros parciales y sistemas que no conversan bien entre sí, la operación pierde continuidad y se vuelve más vulnerable a retrasos, reprocesos y decisiones tardías.

Las causas son varias. Primero, hay una fragmentación operacional evidente. Quien evalúa no administra inventario. Quien ejecuta no valida. Quien compra no necesariamente ve el impacto que su demora genera sobre la OT. Esa desconexión abre cuellos de botella, sobre todo cuando falta stock o cuando la prioridad de una intervención cambia. Segundo, aparece una causa tecnológica igual de seria: el CMMS puede registrar la orden, pero no necesariamente integra los eventos críticos del resto del proceso. Entonces florecen planillas paralelas, seguimientos por mensaje y decisiones basadas en conocimiento tácito. Mala señal. Cuando eso pasa, la información deja de comportarse como un activo del sistema y se convierte en un problema de coordinación entre personas.

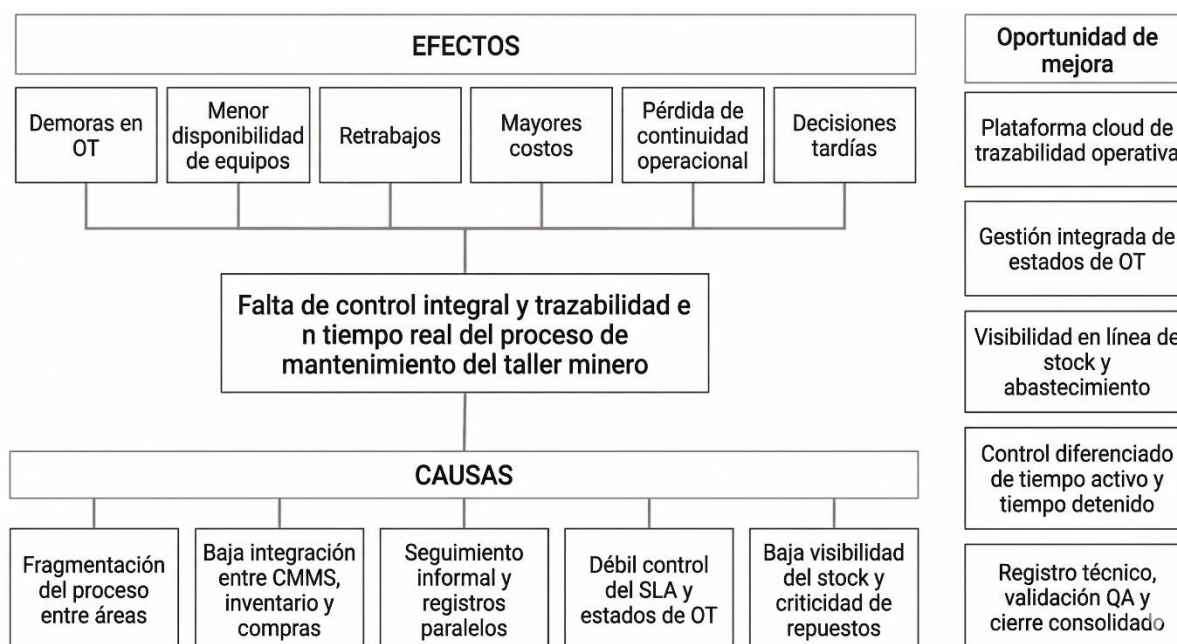


También hay un problema fino, pero decisivo, en la trazabilidad del SLA. El flujo contempla pausas y reanudaciones cuando no hay stock, lo que tiene lógica. El problema aparece cuando esos cambios de estado no quedan registrados con claridad y visibilidad uniforme. Ahí ya no es posible distinguir bien si una demora pertenece a mantenimiento, a abastecimiento o a validación. Lo mismo ocurre con la gestión de inventario: si el stock no está actualizado en línea, si la reserva no queda amarrada a la OT o si compras no recibe contexto suficiente sobre criticidad, la respuesta llega tarde. Y en mantenimiento minero, llegar tarde cuesta caro.

Los efectos son concretos. Baja la disponibilidad de equipos. Aumentan los tiempos muertos. Se deteriora la coordinación entre áreas. Se resiente la continuidad operacional. Suben los

costos por horas improductivas, urgencias logísticas y retrabajos. También crece el riesgo operativo, porque disminuye la capacidad de seguimiento y control sobre la secuencia real del mantenimiento. No es un problema menor. Es una debilidad estructural del proceso.

Ahora bien, esa misma debilidad abre una oportunidad clara. Una solución cloud bien diseñada permitiría transformar un flujo fragmentado en un proceso visible, trazable y coordinado de punta a punta. No solo mejoraría la eficiencia del taller. También elevaría el estándar de gestión, porque convertiría datos dispersos en información útil para actuar mejor y más rápido. Ahí está el valor real del proyecto.



3. Objetivo general y objetivos específicos

3.1 Objetivo general

Diseñar una solución informática cloud para el proceso de mantenimiento del taller minero que integre la gestión de órdenes de trabajo, repuestos, control de SLA, ejecución técnica, validación y cierre, de manera que permita registrar y monitorear el 100% de los hitos críticos del flujo definido y mejorar la trazabilidad operativa del proceso.

3.2 Objetivos específicos

1. Levantar, modelar y analizar el proceso actual de mantenimiento del taller minero, definiendo el proceso mejorado, las reglas de negocio, los requerimientos funcionales y no funcionales, y los puntos de integración con CMMS, bodega y ERP, de forma que el 100% de las transiciones críticas de la OT quede formalmente especificado.
2. Definir una arquitectura cloud preliminar y desarrollar un diseño funcional validado, incorporando modelo de datos inicial, mockup o prototipo navegable, propuesta de integración, estrategia inicial de migración y plan de implementación por etapas, con el fin de demostrar la viabilidad técnica y operativa de la solución propuesta.

Estos objetivos se conectan de forma directa con el problema detectado. El primero se hace cargo del negocio y del proceso. El segundo traduce ese diagnóstico a una propuesta tecnológica concreta. No hay salto al vacío. Primero se entiende el problema; después se diseña la respuesta. Así debe ser.

4. Alcance del proyecto

4.1 Alcance general

El proyecto abarca el análisis, diseño y validación preliminar de una solución informática cloud orientada a mejorar el proceso de mantenimiento de equipos en un taller minero, desde la generación de la OT hasta su cierre técnico-financiero. El alcance incorpora la dimensión de negocio y la dimensión tecnológica. Esto incluye levantar el proceso actual, identificar brechas, proponer un flujo mejorado, definir requerimientos, diseñar una arquitectura cloud preliminar, estructurar un modelo de datos inicial, plantear integraciones con sistemas corporativos y desarrollar un prototipo o mockup que permita visualizar el funcionamiento esperado de la solución.

4.2 Elementos incluidos dentro del proyecto

Dentro del proyecto se consideran: el levantamiento del proceso actual; el diagnóstico del problema; la propuesta de proceso mejorado; la definición de requerimientos funcionales, no funcionales y reglas de negocio; el diseño funcional de la solución; la arquitectura cloud preliminar; el modelo de datos inicial; el prototipo o mockup navegable; la propuesta de integración con CMMS, ERP o inventario; la estrategia inicial de migración; el plan de implementación; y la planificación general del trabajo académico. Cada uno de estos componentes forma parte del proyecto porque aporta evidencia de viabilidad técnica y operativa.

4.3 Componentes considerados pero no implementados completamente en esta etapa

Hay elementos que sí pertenecen al proyecto, pero que en esta etapa no se ejecutarán de manera completa. Entre ellos se encuentran el desarrollo o programación del sistema, la definición de arquitectura cloud en servicios concretos de AWS, Azure o GCP, la estrategia de implementación en contexto minero real, la migración o carga inicial de datos y la integración con CMMS/ERP. Todo eso sí se considera. Lo que cambia es el nivel de profundidad: en esta fase se aborda desde el diseño, la propuesta técnica, la simulación, el prototipo o la validación conceptual, no desde una puesta en producción completa.

4.4 Exclusiones reales del proyecto

Las exclusiones reales son otras. Aquí sí entran actividades que exceden el propósito académico de esta etapa y que no son necesarias para demostrar la viabilidad del proyecto. Entre ellas se encuentran la operación productiva a escala corporativa, el despliegue simultáneo en múltiples talleres o faenas, la personalización completa del ERP corporativo, la ejecución de compras reales a proveedores, la remediación exhaustiva de datos históricos no estructurados y la certificación formal de ciberseguridad, continuidad o cumplimiento normativo corporativo. En otras palabras, no se excluye pensar estas materias; se excluye ejecutarlas por completo en esta fase.

4.5 Entregables del proyecto

Los entregables contemplados son los siguientes: documento de levantamiento del proceso actual, análisis del problema y la oportunidad, propuesta de proceso mejorado, requerimientos preliminares, diseño funcional de la solución, arquitectura cloud preliminar, modelo de datos inicial, mockup o prototipo navegable, propuesta de integración con CMMS/ERP, estrategia preliminar de migración, plan de implementación y planificación del proyecto mediante EDT/WBS y carta Gantt. No son piezas sueltas. Funcionan como un conjunto que permite sostener la propuesta con lógica de negocio y sustento técnico.

4.6 Supuestos

Para desarrollar el proyecto se asume que la organización cuenta con conectividad suficiente para operar una solución cloud en el taller y en las áreas administrativas involucradas; que existe un CMMS operativo y algún mecanismo formal para gestionar inventario y compras; que habrá disponibilidad de usuarios clave para validar requerimientos; que existen catálogos mínimos de equipos, repuestos, usuarios y OT; y que la implementación, en un escenario futuro, sería gradual y conviviría con sistemas legados. Son supuestos razonables. También son necesarios para poder diseñar con un piso mínimo de realismo.

4.7 Restricciones

El proyecto también enfrenta restricciones. La principal es el tiempo acotado propio de una etapa académica, lo que obliga a priorizar análisis, diseño y validación preliminar por sobre una implementación completa. Se suma el acceso limitado a datos reales, la dependencia de sistemas legados, las restricciones organizacionales de seguridad TI y la disponibilidad parcial de actores del negocio. Ninguna de estas limitaciones invalida la propuesta, pero sí condiciona la profundidad con que algunos componentes pueden desarrollarse en esta fase.

4.8 Relación del alcance con los objetivos

El alcance está alineado de forma directa con los objetivos del proyecto. El levantamiento del proceso, el análisis del problema, la definición de requerimientos y el rediseño del flujo responden al objetivo enfocado en comprender y formalizar el proceso. La arquitectura cloud, el modelo de datos, el prototipo, la propuesta de integración, la estrategia de migración y el plan de implementación responden al objetivo orientado a demostrar viabilidad técnica. Así, el proyecto no queda reducido a una reflexión teórica ni se vende humo con una falsa promesa de implementación total. Queda donde debe quedar: como una solución cloud viable, bien formulada y desarrollable por etapas.

5. Planificación de actividades

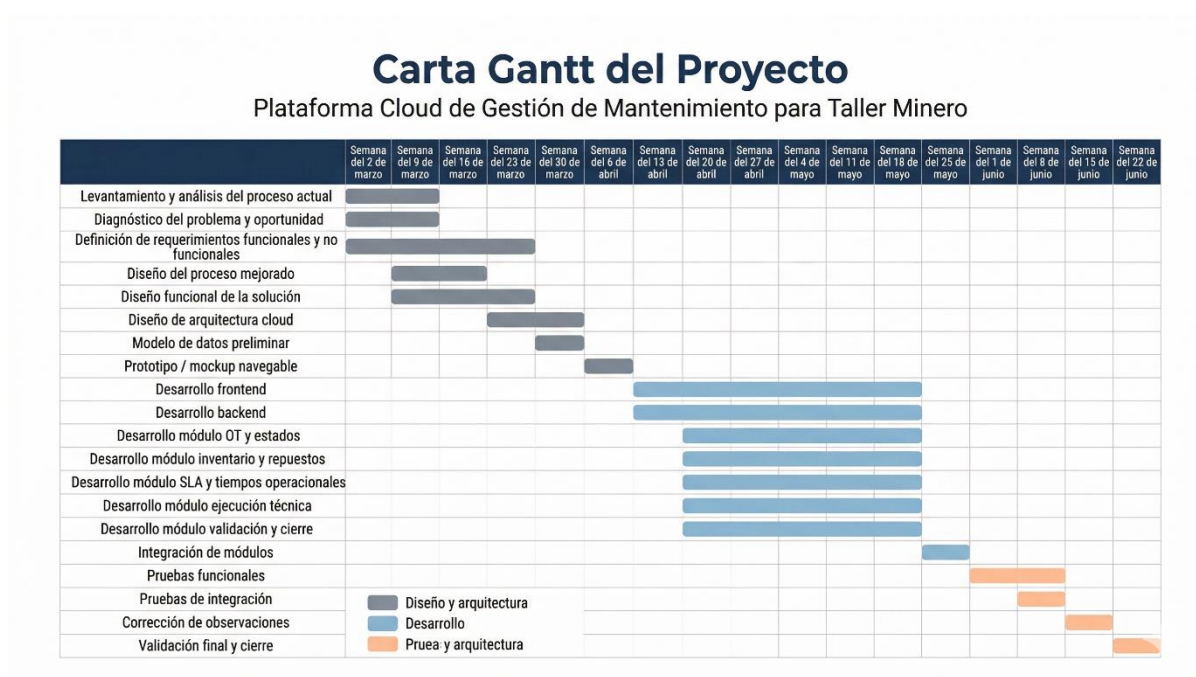
5.1 EDT / WBS del proyecto

La estructura de trabajo del proyecto se organiza en tres grandes fases: inicio, desarrollo y cierre. La fase de inicio considera la definición del proyecto y la organización del equipo. La fase de desarrollo abarca el levantamiento del proceso actual, el diagnóstico del problema, la formulación de objetivos, la definición del alcance, el levantamiento preliminar de requerimientos, el diseño del proceso mejorado, el diseño funcional de la solución cloud, la arquitectura, el modelo de datos, la propuesta de integración, el prototipo y la estrategia de migración e implementación. La fase de cierre contempla la consolidación del informe, la revisión final y la entrega del avance. La secuencia tiene sentido. No parte por tecnología. Parte por entender el problema.

5.2 Asignación de responsables por rol

La planificación distribuye responsabilidades entre cuatro roles principales. El líder de proyecto coordina el avance y resguarda la coherencia metodológica. El analista de procesos levanta y modela el flujo actual, identifica problemas y define requerimientos. El responsable técnico/cloud diseña la solución, la arquitectura, el modelo de datos, la integración y el prototipo. El responsable documental consolida el informe, revisa la redacción, ordena la estructura y prepara la entrega final. Esta división no solo organiza el trabajo; también evita superposiciones y ayuda a mantener foco en cada frente.

5.3 Carta Gantt simplificada por semanas



5.4 Secuencia lógica de trabajo

La secuencia propuesta responde a una formulación seria de proyecto TI. Parte con una fase de ordenamiento inicial, sigue con la comprensión del proceso y del problema, luego define objetivos y alcance, y solo después aterriza la solución tecnológica. Ese orden importa. Mucho. Cuando se invierte, el proyecto termina diseñado desde la herramienta y no desde la necesidad real del negocio. Aquí ocurre lo contrario, y eso le da solidez a la propuesta. La tecnología no aparece como adorno ni como entusiasmo vacío; aparece como respuesta concreta a un problema operativo específico del taller minero.

6. Determinación de tecnologías y servicios cloud

6.1 Stack tecnológico seleccionado

La elección del stack tecnológico no parte desde la preferencia personal ni desde la moda del momento. Parte desde el problema. El sistema necesita gestionar un flujo transaccional complejo, con múltiples roles, estados concurrentes, trazabilidad de tiempos y evidencia documental. Eso define los requisitos mínimos que la tecnología debe cumplir: rendimiento en operaciones CRUD intensivas, control de acceso granular por rol, persistencia relacional robusta, capacidad de operar desde terreno y soporte para despliegue continuo sin fricción.

Con eso como marco, el stack propuesto es el siguiente:

Frontend — Next.js sobre Vercel (PaaS)

Next.js es un framework React de nivel producción que permite renderizado híbrido (SSR y SSG), enrutamiento basado en sistema de archivos, optimización automática de imágenes y soporte nativo para API routes. En el contexto de este proyecto, su ventaja principal es doble. Primero, permite construir una interfaz responsiva que funcione tanto en escritorio (para el Jefe de Taller o el Administrador IT) como en dispositivos móviles o tablets (para el mecánico en terreno). Segundo, al desplegarse sobre Vercel, se obtiene un entorno PaaS con despliegue automático desde repositorio Git, CDN global, dominio HTTPS sin configuración manual y escalado bajo demanda. Vercel ofrece una capa gratuita con ancho de banda y builds suficientes para un proyecto académico y un MVP funcional.

Backend — NestJS sobre Railway (PaaS)

NestJS es un framework de Node.js construido sobre Express (o Fastify), diseñado con arquitectura modular, inyección de dependencias y soporte nativo para TypeScript. En un sistema como este, donde el flujo de la OT cruza múltiples dominios (mantenimiento, inventario, abastecimiento, calidad), la modularidad de NestJS permite organizar el backend en módulos desacoplados: módulo de OT, módulo de inventario, módulo de SLA, módulo de usuarios y roles, entre otros. Eso no es un detalle menor. Es lo que permite escalar la solución sin convertir el código en un monolito inmanejable. Railway, por su parte, ofrece un entorno PaaS que facilita el despliegue de aplicaciones Node.js con variables de entorno seguras, logs en tiempo real, dominio público automático y una capa gratuita que incluye 500 horas de ejecución mensuales y 1 GB de RAM, suficiente para el alcance del proyecto.

Base de datos y servicios complementarios — Supabase (BaaS/PaaS)

Supabase es una plataforma open source construida sobre PostgreSQL que ofrece base de datos relacional, autenticación, almacenamiento de archivos y funciones en tiempo real desde un solo servicio. Para este proyecto, la elección de Supabase no es arbitraria. El modelo de datos del CMMS es intrínsecamente relacional: una OT se vincula a un equipo, a un mecánico, a múltiples repuestos, a eventos de SLA y a registros de validación. PostgreSQL maneja esas relaciones con integridad referencial, transacciones ACID y capacidad de consultas complejas que una base NoSQL no garantiza con la misma solidez. Además, Supabase incluye autenticación con control de roles mediante Row Level Security (RLS), lo cual permite implementar de forma nativa los perfiles de Jefe de Taller, Mecánico, Bodeguero y Administrador IT sin depender de librerías externas. El almacenamiento integrado (Supabase Storage) resuelve la necesidad de registrar evidencia fotográfica durante la

ejecución del mantenimiento. Su capa gratuita ofrece 500 MB de base de datos, 1 GB de almacenamiento de archivos, 50.000 filas de autenticación y acceso completo a la API REST y en tiempo real.

6.2 Clasificación de servicios cloud utilizados

El stack propuesto opera íntegramente sobre servicios cloud clasificados de la siguiente forma:

PaaS (Platform as a Service): Vercel y Railway. Ambas plataformas abstraen la infraestructura subyacente (servidores, redes, sistema operativo) y permiten al equipo concentrarse exclusivamente en el código de la aplicación. No se administran máquinas virtuales ni contenedores de forma manual. El desarrollador sube código, la plataforma compila, despliega y escala. Esa es la propuesta de valor del modelo PaaS y es coherente con un equipo académico que necesita eficiencia operativa sin sobrecarga de administración de infraestructura.

BaaS (Backend as a Service) / PaaS: Supabase. Aunque se clasifica frecuentemente como BaaS porque provee backend funcional listo para usar (autenticación, base de datos, storage, funciones), su núcleo es un PostgreSQL gestionado que también puede entenderse como PaaS de base de datos. En la práctica, Supabase entrega tanto la persistencia como servicios complementarios que normalmente requerirían desarrollo adicional.

SaaS (Software as a Service): GitHub como plataforma de control de versiones y colaboración, y Linear como herramienta de gestión de tickets y seguimiento de tareas del equipo. Ambas se consumen como servicio sin intervención en su infraestructura ni personalización de su código fuente.

No se utiliza IaaS (Infrastructure as a Service) de forma directa. No hay servidores virtuales, máquinas EC2 ni configuración de redes o balanceadores. Esa decisión es deliberada: el modelo PaaS/BaaS reduce la complejidad operativa, acelera el despliegue y permite que el equipo se enfoque en resolver el problema de negocio, no en administrar infraestructura.

7. Justificación de la elección tecnológica

7.1 Por qué cloud y no on-premise

La naturaleza del problema lo exige. El proceso de mantenimiento del taller minero involucra actores que operan en distintos espacios físicos y en distintos momentos: el Administrador IT en oficina, el Jefe de Taller en la zona de evaluación, el Bodeguero en el área de inventario y el Mecánico en terreno, muchas veces en condiciones donde no hay un escritorio fijo ni acceso a una red corporativa interna. Una solución on-premise obligaría a VPN, infraestructura local, servidores físicos y un equipo de TI dedicado a mantenerlo. En una operación minera, donde la conectividad ya es un desafío, eso es agregar fricción, no resolverla.

El modelo cloud elimina esa barrera. Cualquier usuario con conexión a internet y un navegador accede al sistema. No hay instalación de software cliente. No hay dependencia de un servidor físico dentro de la faena. Y si mañana se necesita operar desde otro taller o desde una oficina remota, la solución ya está preparada para eso sin rediseño.

7.2 Justificación de cada componente del stack

Next.js + Vercel — Justificación del frontend

Se evaluaron alternativas como React con Vite (SPA puro) y Angular. React + Vite es una opción válida, pero carece de renderizado del lado del servidor (SSR), lo que limita la velocidad de carga inicial y el SEO si en algún momento el sistema requiere un portal externo. Angular ofrece robustez, pero su curva de aprendizaje es mayor y su ecosistema de despliegue no está tan integrado como el par Next.js + Vercel. La combinación elegida ofrece SSR cuando se necesita, generación estática cuando conviene, API routes para lógica ligera y un pipeline de despliegue que va desde el push al repositorio hasta producción en menos de un minuto. Para un equipo académico con plazos ajustados, esa velocidad de iteración no es un lujo: es una necesidad.

NestJS + Railway — Justificación del backend

Se consideraron Express puro, FastAPI (Python) y Django. Express es flexible pero no impone estructura, lo que en un proyecto con múltiples dominios de negocio (OT, inventario, SLA, usuarios, QA) tiende a generar código desordenado a medida que crece. FastAPI es excelente para APIs rápidas, pero el equipo tiene mayor dominio en el ecosistema JavaScript/TypeScript, y mantener un solo lenguaje en frontend y backend reduce la fricción de desarrollo y facilita la reutilización de tipos y validaciones. Django es robusto pero pesado para una API REST pura, y su modelo de templates no aporta valor cuando el frontend ya está resuelto con Next.js. NestJS ocupa un punto intermedio muy preciso: impone arquitectura modular, usa TypeScript de forma nativa, soporta inyección de dependencias y se integra bien con ORMs como Prisma o TypeORM para conectar con PostgreSQL. Railway se eligió sobre alternativas como Render o Fly.io porque ofrece un entorno de despliegue simple, con soporte nativo para Node.js, variables de entorno seguras, logs accesibles y una capa gratuita que cubre el alcance del proyecto sin requerir tarjeta de crédito para comenzar.

Supabase — Justificación de la base de datos

Se evaluaron Neon (PostgreSQL serverless), MongoDB Atlas y Turso (SQLite distribuido). MongoDB Atlas fue descartada porque el modelo de datos del CMMS es profundamente relacional: una OT se vincula a equipos, mecánicos, repuestos, eventos de SLA y registros de validación con integridad referencial estricta. Un esquema documental no garantiza esa consistencia sin lógica adicional en la aplicación. Neon es una alternativa seria, con PostgreSQL serverless y capa gratuita competitiva, pero entrega exclusivamente la base de datos. Para autenticación, almacenamiento de archivos y funciones en tiempo real habría que integrar servicios adicionales como Auth0, Cloudinary o Firebase, lo que multiplica proveedores y complejidad de configuración. Turso es una tecnología prometedora basada en libSQL, pero su ecosistema aún es joven y su documentación no alcanza la madurez de PostgreSQL. Supabase resuelve todo desde una sola plataforma: base de datos PostgreSQL con RLS para control de acceso por rol, autenticación integrada, almacenamiento de archivos para evidencia fotográfica y suscripciones en tiempo real para actualizar estados de OT en vivo. Esa consolidación de servicios reduce la deuda técnica y simplifica la operación.

Linear — Justificación de la gestión de proyecto

Se eligió Linear por sobre Jira o Trello porque ofrece una experiencia orientada a equipos de desarrollo de software con flujos ágiles, sin la sobrecarga de configuración que impone Jira ni la limitación estructural de Trello para gestionar sprints, prioridades y dependencias entre tareas. Linear permite organizar el trabajo en ciclos, asignar responsables, establecer prioridades y mantener visibilidad del avance del proyecto de forma limpia y eficiente.

7.3 Factibilidad de cumplimiento

Factibilidad técnica. Todas las tecnologías seleccionadas son de código abierto o tienen capas gratuitas funcionales. NestJS, Next.js y Supabase cuentan con documentación extensa, comunidades activas y ecosistemas maduros. El equipo tiene experiencia en TypeScript, lo que permite operar frontend y backend con un solo lenguaje, reduciendo la curva de aprendizaje y los errores de integración.

Factibilidad operativa. El modelo PaaS/BaaS elimina la necesidad de administrar infraestructura. No se requiere aprovisionar servidores, configurar balanceadores ni gestionar certificados SSL. Vercel, Railway y Supabase manejan eso de forma transparente. Eso libera al equipo para concentrarse en el desarrollo funcional de la solución, lo cual es crítico dado el tiempo acotado de la etapa académica.

Factibilidad económica. El costo del stack en su fase inicial es cero. Vercel ofrece capa gratuita con builds y ancho de banda suficientes. Railway provee 500 horas mensuales de ejecución sin costo. Supabase entrega 500 MB de base de datos, 1 GB de storage y autenticación ilimitada en su plan gratuito. Linear ofrece plan gratuito para equipos pequeños. GitHub es gratuito para repositorios públicos y privados. En un escenario de escalamiento futuro, los costos de estos servicios crecen de forma proporcional al uso, sin inversión inicial de capital.

8. Conceptualización de la solución

8.1 Propósito de la solución

El propósito de la solución es transformar un proceso de mantenimiento que hoy opera de forma fragmentada en un flujo digital integrado, trazable y gobernable desde una única plataforma cloud. No se trata de automatizar por automatizar. Se trata de resolver un problema concreto: la pérdida de visibilidad, coordinación y control que ocurre cuando la información del proceso queda dispersa entre sistemas que no conversan, registros paralelos y comunicaciones informales.

La plataforma propuesta permite registrar, seguir y controlar el ciclo completo de una Orden de Trabajo, desde su creación en el CMMS hasta su cierre técnico-financiero, pasando por evaluación, gestión de repuestos, asignación, ejecución, validación de calidad y liberación del equipo. Cada transición queda documentada con responsable, fecha, observaciones y evidencia. Eso significa que cualquier actor del proceso puede saber en cualquier momento en qué estado está una OT, quién la tiene, por qué está detenida y qué falta para avanzar.

8.2 Valor que agrega al cliente

El valor no se mide solo en funcionalidades. Se mide en lo que el cliente deja de perder y en lo que empieza a ganar.

En lo operativo, el cliente gana trazabilidad completa del flujo de mantenimiento. Cada OT tiene una línea de tiempo verificable. Los estados son normalizados. Las pausas por falta de stock quedan registradas y separadas del tiempo efectivo de intervención. Eso permite medir de verdad el desempeño del taller, sin contaminar los indicadores con tiempos que no dependen del equipo de mantenimiento.

En lo táctico, el Jefe de Taller gana capacidad de decisión. Puede priorizar trabajos con información real, no con intuición. Puede ver qué mecánico tiene carga disponible, qué OT lleva más tiempo detenida y qué repuestos están pendientes de recepción.

En lo estratégico, la gerencia de mantenimiento obtiene datos consolidados para análisis de disponibilidad de flota, costos por OT, frecuencia de fallas por equipo, tiempos promedio de resolución y cumplimiento de SLA. Esos indicadores hoy no existen de forma confiable, o existen con tanto esfuerzo manual que pierden oportunidad. La solución los genera como subproducto natural del proceso.

En lo económico, la reducción de tiempos muertos, la mejor coordinación con bodega y compras, la disminución de retrabajos por falta de validación y la eliminación de seguimientos informales impactan directamente en los costos operativos del taller.

8.3 Atributos de calidad de la solución

La solución se diseña considerando los siguientes atributos de calidad, alineados con las mejores prácticas de la industria del software y con estándares como ISO/IEC 25010 (modelo de calidad de producto de software):

Integridad

La integridad se refiere a la protección de los datos contra modificaciones no autorizadas, inconsistencias o pérdidas. En esta solución, la integridad se garantiza en múltiples niveles. A nivel de base de datos, PostgreSQL opera con transacciones ACID, lo que asegura que cada operación se ejecute de forma atómica, consistente, aislada y durable. A nivel de modelo de datos, las relaciones entre entidades (OT, equipos, repuestos, mecánicos, eventos de SLA) se implementan con claves foráneas e integridad referencial, lo que impide registros huérfanos o vínculos rotos. A nivel de aplicación, NestJS permite implementar validaciones de entrada mediante DTOs (Data Transfer Objects) con class-validator, rechazando datos malformados antes de que lleguen a la base de datos.

Confiabilidad

La confiabilidad se refiere a la capacidad del sistema de operar de forma consistente y sin fallas bajo condiciones normales de uso. Vercel y Railway operan sobre infraestructura distribuida con alta disponibilidad. Supabase ofrece backups automáticos diarios de la base de datos incluso en su capa gratuita. NestJS, al imponer arquitectura modular, reduce el acoplamiento entre componentes, lo que significa que un fallo en el módulo de inventario no debería afectar el módulo de OT. Además, el uso de TypeScript en todo el stack reduce errores en tiempo de ejecución gracias al tipado estático, detectando inconsistencias antes del despliegue.

Precisión

La precisión se refiere a que la información que el sistema entrega sea exacta, oportuna y con el nivel de detalle adecuado para cada usuario. En esta solución, la precisión se trabaja en dos frentes. Primero, en la captura del dato: cada registro de estado, hora, responsable y observación se genera en el momento en que ocurre la acción, vinculado al usuario autenticado y con marca de tiempo del servidor. Segundo, en la entrega del dato: cada rol ve la información que le corresponde con el nivel de detalle que necesita. El mecánico ve su OT asignada, los repuestos disponibles y el protocolo a seguir. El Jefe de Taller ve el estado general del taller, las OT pendientes de validación y los indicadores de rendimiento. Esa diferenciación no es cosmética. Es precisión funcional.

Oportunidad

La oportunidad se refiere a que la información esté disponible en el momento en que se necesita, no después. La solución aborda la oportunidad mediante tres mecanismos. Primero, Supabase Realtime permite suscribirse a cambios en tablas específicas, de modo que cuando una OT cambia de estado, los usuarios relevantes reciben la actualización sin necesidad de refrescar la pantalla. Segundo, Next.js con SSR permite cargar dashboards con datos actualizados desde el servidor en cada acceso. Tercero, el sistema de notificaciones y alertas permite avisar al Jefe de Taller cuando un repuesto ingresa a bodega, al mecánico cuando se le asigna una OT, o al Administrador cuando un SLA está próximo a vencer. La información llega al actor correcto en el momento correcto.

Seguridad

La seguridad se implementa en capas. A nivel de autenticación, Supabase Auth gestiona el inicio de sesión con tokens JWT, soporta autenticación por correo y contraseña, y permite integrar proveedores OAuth. A nivel de autorización, PostgreSQL con Row Level Security (RLS) restringe el acceso a los datos según el rol del usuario autenticado. Un mecánico no

puede modificar una OT que no le fue asignada. Un bodeguero no puede aprobar una validación QA. Esas restricciones viven en la base de datos, donde son inviolables desde la aplicación. A nivel de transporte, todas las comunicaciones viajan sobre HTTPS con certificados SSL gestionados automáticamente. A nivel de configuración, las credenciales se almacenan como variables de entorno, nunca en el código fuente. Estas prácticas son consistentes con los principios recomendados por OWASP.

8.4 Viabilidad de la solución

La solución es viable porque cumple tres condiciones simultáneas. Es técnicamente factible: las tecnologías existen, son maduras, están documentadas y el equipo tiene las competencias para usarlas. Es operativamente factible: el modelo PaaS/BaaS elimina la carga de administración de infraestructura y permite iterar rápido dentro del plazo académico. Y es económicamente factible: el costo inicial es cero y el crecimiento futuro es proporcional al uso. No se promete un sistema en producción. Se promete un diseño sólido, validado con prototipo, sustentado en tecnología real y alineado con el problema del taller minero.

9. Metodología, ciclo de vida, patrón de diseño y mecanismos de control

9.1 Metodología seleccionada: Scrum adaptado al contexto académico

La metodología seleccionada es Scrum, adaptada a las condiciones reales del proyecto. Scrum es un marco de trabajo ágil diseñado para entregar valor de forma incremental mediante ciclos cortos de trabajo llamados sprints, con ceremonias definidas de planificación, revisión y retroalimentación. No se eligió por popularidad. Se eligió porque responde a tres condiciones concretas del proyecto.

Primero, el problema no está completamente definido desde el inicio. El proceso de mantenimiento del taller minero tiene una lógica general clara, pero los detalles funcionales, las reglas de negocio específicas y las prioridades de los usuarios emergen a medida que se profundiza en el análisis. Un enfoque predictivo como cascada exigiría definir todo al principio, lo cual no es realista cuando el conocimiento del dominio se construye progresivamente. Scrum acepta esa incertidumbre y la gestiona mediante iteraciones.

Segundo, el equipo es pequeño (tres integrantes) y el plazo es acotado. Scrum funciona bien en equipos reducidos porque distribuye responsabilidad sin burocracia excesiva. No requiere documentación exhaustiva entre fases, sino artefactos funcionales: backlog, incrementos y revisiones periódicas.

Tercero, la solución es cloud y modular. NestJS permite construir el backend por módulos independientes (OT, inventario, SLA, usuarios, QA), y Next.js permite desarrollar interfaces por funcionalidad. Esa arquitectura se presta naturalmente para entregas incrementales: primero el módulo de OT con su interfaz, luego inventario, luego SLA, y así sucesivamente.

Adaptaciones al contexto académico

Los sprints se definen en ciclos de dos semanas, alineados con los hitos de entrega académica. El rol de Product Owner lo asume el líder del equipo, quien prioriza el backlog en función de los indicadores de logro de la evaluación y los objetivos del proyecto. No hay Scrum Master dedicado; esa función se distribuye entre los integrantes. Las dailies se reemplazan por reuniones de sincronización dos veces por semana. El backlog se gestiona en Linear, donde cada tarea tiene asignado un responsable, una prioridad, un estado y un ciclo asociado. Estas adaptaciones no desvirtúan Scrum. Lo hacen operativo para un equipo de tres personas con dedicación parcial y plazos fijos.

9.2 Ciclo de vida del proyecto: Iterativo-incremental

El ciclo de vida adoptado es iterativo-incremental. Un ciclo iterativo significa que el producto se desarrolla en pasadas sucesivas, donde cada iteración refina y mejora lo construido anteriormente. Un ciclo incremental significa que cada iteración agrega funcionalidad nueva al sistema. La combinación permite avanzar en dos frentes: profundizar lo que ya existe y ampliar lo que falta.

El ciclo se estructura en cuatro incrementos principales:

Incremento 1 — Núcleo de OT y usuarios. Comprende el modelo de datos base, la autenticación por roles (Supabase Auth + RLS), el CRUD de órdenes de trabajo y la interfaz

de gestión de OT. Al finalizar este incremento, el sistema permite crear una OT, asignarla y cambiar su estado.

Incremento 2 — Inventario y gestión de repuestos. Agrega el módulo de inventario, la consulta de stock, la reserva de repuestos vinculada a una OT y la lógica de pausa de SLA cuando no hay disponibilidad.

Incremento 3 — Ejecución, validación y QA. Incorpora el registro de ejecución del mecánico (horas, protocolo LOTO, evidencia fotográfica vía Supabase Storage), la validación del Jefe de Taller y el circuito de retrabajo.

Incremento 4 — Cierre, reportería y refinamiento. Agrega el cierre técnico-financiero, los dashboards de indicadores (disponibilidad, cumplimiento SLA, costos por OT) y las mejoras de usabilidad.

9.3 Patrón de diseño adoptado: Arquitectura modular por capas

Capa de presentación (Frontend — Next.js). Responsable de la interfaz de usuario, la navegación, la gestión de estado del cliente y la comunicación con el backend mediante llamadas HTTP a la API REST. Cada módulo funcional tiene sus propias páginas y componentes, organizados por dominio.

Capa de lógica de negocio (Backend — NestJS). Responsable de las reglas de negocio, la validación de datos, la orquestación de flujos y la gestión de estados. NestJS implementa esta capa mediante módulos desacoplados, cada uno con sus controladores, servicios y repositorios. La inyección de dependencias nativa facilita el testing unitario y la evolución independiente de cada dominio.

Capa de persistencia (Base de datos — Supabase/PostgreSQL). Responsable del almacenamiento, la integridad referencial, el control de acceso a nivel de fila (RLS) y las consultas. El acceso desde NestJS se realiza mediante Prisma, que permite definir el esquema de datos como código, generar migraciones versionadas y operar con tipos de TypeScript generados automáticamente.

Capa de servicios transversales (Supabase Auth, Storage, Realtime). Estos servicios no pertenecen a un solo módulo. La autenticación se consume desde el frontend y se valida en el backend. El almacenamiento de archivos se usa desde el módulo de ejecución. Las suscripciones en tiempo real se consumen desde los dashboards.

9.4 Mecanismos de control y gestión del proyecto

Control de versiones con GitHub. Todo el código fuente se aloja en un repositorio Git con ramas por funcionalidad. Los cambios se integran mediante pull requests con revisión cruzada entre integrantes. El historial de commits funciona como registro verificable de contribución individual.

Gestión de tareas con Linear. Cada actividad se registra como un ticket con responsable asignado, prioridad definida, estado actualizado y ciclo asociado. Al cierre de cada ciclo, el equipo revisa el avance y ajusta el backlog del siguiente sprint.

Revisiones de sprint. Al finalizar cada sprint de dos semanas, el equipo presenta el incremento funcional logrado, identifica impedimentos, evalúa la calidad del entregable parcial y decide si es necesario ajustar prioridades.

Despliegue continuo. Tanto Vercel como Railway despliegan automáticamente cada vez que se integra código a la rama principal. Cada integración es una verificación inmediata. Si algo se rompe, se detecta en minutos, no en semanas.

Documentación progresiva. La documentación se construye de forma progresiva a lo largo de los sprints, vinculada a los entregables definidos en el alcance. Al cierre del proyecto, la documentación es un reflejo fiel del proceso, no una reconstrucción apresurada.

10. Documentación, revisiones y estándares de calidad

10.1 Enfoque de documentación del proyecto

La documentación de este proyecto no se concibe como una actividad de cierre. Se concibe como un proceso continuo que acompaña cada fase del desarrollo. Cada entrega parcial genera su propio registro documental, y cada revisión queda respaldada con evidencia verificable. Esa lógica responde a un principio básico de gestión de proyectos: si no está documentado, no existe.

El enfoque adoptado se alinea con las prácticas recomendadas por el PMBOK en lo que respecta a gestión de alcance, tiempo, calidad y comunicaciones, y con los principios del manifiesto ágil en cuanto a priorizar software funcionando sobre documentación exhaustiva, sin que eso signifique no documentar. La documentación sirve al proyecto, no al revés.

10.2 Estructura documental por entrega parcial

Estado de avance N°1 — Formulación y análisis. Corresponde a la presente entrega. Incluye el contexto del proyecto, la identificación del problema u oportunidad, los objetivos general y específicos, el alcance con entregables, supuestos y restricciones, la planificación de actividades con EDT y carta Gantt, la determinación del stack tecnológico y servicios cloud, la justificación de la elección tecnológica, la conceptualización de la solución con atributos de calidad, la definición de metodología, ciclo de vida, patrón de diseño y mecanismos de control, y la presente sección de documentación. Este documento constituye la línea base del proyecto.

Estado de avance N°2 — Diseño y prototipado (proyectado). Incluirá el modelo de datos detallado con diagrama entidad-relación, los requerimientos funcionales y no funcionales formalizados, las reglas de negocio documentadas, el diseño de la arquitectura cloud con diagrama de componentes, los mockups o prototipo navegable, la propuesta de integración con CMMS/ERP y la estrategia preliminar de migración.

Entrega final — Consolidación y validación (proyectado). Consolidará toda la documentación anterior, incorporará el prototipo funcional desplegado, los resultados de las pruebas de validación, las observaciones recogidas en las revisiones de sprint, las lecciones aprendidas y las recomendaciones para una fase de implementación futura.

10.3 Registro de revisiones y certificación de avance

Acta de revisión de sprint. Documento breve que registra la fecha de la revisión, los integrantes presentes, el incremento presentado, las observaciones del equipo, los impedimentos detectados y los acuerdos para el siguiente ciclo. Estas actas se almacenan en el repositorio del proyecto.

Transcripciones de reuniones. Cada reunión de sincronización y revisión de sprint se registra mediante transcripción, ya sea generada automáticamente por la herramienta de videollamada utilizada o redactada por el integrante designado como relator de la sesión. Las transcripciones incluyen fecha, participantes, temas tratados, decisiones tomadas y tareas derivadas. Este registro cumple una función doble: permite a cualquier integrante que no haya podido asistir reconstruir lo discutido sin depender de versiones informales, y constituye evidencia verificable de la dinámica de trabajo del equipo, algo especialmente relevante

considerando que la calificación es individual. Las transcripciones se almacenan en la carpeta /docs/reuniones del repositorio, nombradas con la convención YYYY-MM-DD_tipo-reunion.md.

Checklist de avance por incremento. Para cada incremento definido en el ciclo de vida, se mantiene un checklist que detalla los entregables comprometidos, su estado de completitud y las observaciones asociadas. Este instrumento permite verificar de forma objetiva si el avance es real.

Registro de cambios (changelog). Cada modificación relevante en el alcance, la arquitectura, el modelo de datos o las decisiones técnicas se documenta con fecha, descripción del cambio, justificación y responsable.

Historial de commits y pull requests. El repositorio en GitHub funciona como un registro automático y verificable de toda la actividad de desarrollo. Cada commit tiene fecha, autor y descripción. Cada pull request tiene revisión cruzada y aprobación antes de integrarse.

10.4 Estándares de calidad aplicados

PMBOK (Project Management Institute). Se aplican prácticas del PMBOK en la definición del alcance, la estructura de desglose del trabajo (EDT/WBS), la planificación temporal mediante carta Gantt, la gestión de supuestos y restricciones, y los mecanismos de control de cambios.

ISO/IEC 25010 — Modelo de calidad de producto de software. Los atributos de calidad documentados en la conceptualización (integridad, confiabilidad, precisión, oportunidad y seguridad) se derivan de este estándar internacional.

Manifiesto Ágil y Guía Scrum. La metodología adoptada se basa en los valores y principios del manifiesto ágil, y la implementación de Scrum sigue las directrices de la Scrum Guide. Las adaptaciones realizadas están documentadas y justificadas.

OWASP (Open Web Application Security Project). Las prácticas de seguridad descritas (autenticación JWT, Row Level Security, HTTPS, variables de entorno para secretos) son consistentes con las recomendaciones de OWASP para aplicaciones web.

Convenciones de desarrollo. El proyecto adopta TypeScript estricto, linting con ESLint, formato con Prettier y estructura de carpetas estandarizada tanto en NestJS como en Next.js.

10.5 Trazabilidad entre documentación y entregables

Para garantizar que la documentación no quede desconectada de los entregables reales, se establece una matriz de trazabilidad que vincula cada sección del informe con el artefacto que la respalda. El contexto del proyecto y el modelo de solución se respaldan con el diagrama BPMN del proceso y el documento de análisis. La identificación del problema se respalda con el diagrama de Ishikawa o árbol de oportunidad. Los objetivos y el alcance se respaldan con la EDT/WBS y la carta Gantt. La determinación tecnológica se respalda con el diagrama de arquitectura cloud y la tabla comparativa de tecnologías. La conceptualización se respalda con la documentación de atributos de calidad y el mapeo a ISO 25010. La metodología se respalda con el backlog en Linear, las actas de sprint, las transcripciones de

reuniones y el historial de commits. El prototipo se respalda con el despliegue funcional en Vercel y Railway, accesible vía URL.

10.6 Gestión de la documentación

Toda la documentación del proyecto se gestiona bajo las siguientes reglas. Los documentos se almacenan en el repositorio GitHub del proyecto, dentro de una carpeta dedicada /docs, versionada junto con el código. Los documentos se redactan en formato Markdown para facilitar la edición colaborativa, el versionado con Git y la visualización directa en GitHub. Para las entregas formales, se exportan a formato Word o PDF según los requisitos de la evaluación. Las versiones entregadas se etiquetan (tags) en el repositorio con la fecha y el nombre de la entrega, lo que permite reconstruir el estado exacto del proyecto en cualquier punto del tiempo.